



TRIBHUVAN UNIVERSITY

Prime College

Nayabazar, Kathmandu, Nepal

A Project Report

On

“DeNoisify:

Intelligent Audio Denoising System”

Submitted By:

Bibek Tiwari (28551/078)

Kismat Roka (28558/078)

Min Raj Panday (28560/078)

A Project Report Submitted in partial fulfillment of the requirement of **Bachelor of Science in Computer Science & Information Technology (BSc.CSIT) 7th Semester** of Tribhuvan University, Nepal

Ashoj, 2082

DeNoisify : Intelligent Audio Denoising System

[CSC 412]

A project report submitted for the partial fulfillment of the requirement for the degree of Bachelor of Science in Computer science & Information Technology awarded by Tribhuvan University.

Submitted By

Bibek Tiwari (28551/078)

Kismat Roka (28558/078)

Min Raj Panday (28560/078)

Submitted To

Prime College

Department of Computer science

Affiliated to Tribhuvan University

Khusibun, Nayabazar, Kathmandu



Ashoj, 2082

Date: 2nd Ashoj, 2082

SUPERVISOR’S RECOMMENDATION

It is my pleasure to recommend that a report on “**DENOISIFY : INTELLIGENT AUDIO DENOISING SYSTEM**” has been prepared under my supervision by Bibek Tiwari, Kismat Roka and Min Raj Panday in partial fulfillment of the requirement of the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT). Their report is satisfactory to process for the future evaluation.

.....

Mr. Sudan Prajapati

Supervisor

Department of Computer Science & IT

Prime College

Date: 2nd Ashoj, 2082

CERTIFICATE OF APPROVAL

This is to certify that this report has been read and recommended to the Department of Computer Science and Information Technology for acceptance of report entitled **“DENOISIFY : INTELLIGENT AUDIO DENOISING SYSTEM”** submitted by Bibek Tiwari, Kismat Roka, Min Raj Panday in partial fulfillment for the degree of Bachelor of Science in Computer Science and Information Technology (BSc.CSIT), Institute of Science and Technology, Tribhuvan University.

.....

Ms. Rolisha Sthapit

Vice Principal

.....

Mr. Sailesh Karmacharya

Research Coordinator

.....

Mr. Sudan Prajapati

Supervisor

.....

External Examiner

ACKNOWLEDGEMENT

We would like to extend our sincere gratitude to all those who contributed to the successful completion of this report. In particular, we express our deep appreciation to **Mr. Sudan Prajapati**, our project supervisor, for his invaluable guidance, thoughtful recommendations, and unwavering support, which greatly facilitated the coordination of our project, especially in the preparation of this report.

We are also grateful to the expert panels and our secondary supervisor for their constructive advice during our project presentation. Their feedback and insightful suggestions significantly enhanced the quality of our work.

Our heartfelt thanks go to our colleagues for their continuous encouragement and assistance throughout the project. Their support was instrumental in its completion.

Finally, we would like to thank everyone who contributed to the successful completion of this project. Your help, support, and motivation have been deeply appreciated.

With respect,

Bibek Tiwari (28551/078)

Kismat Roka (28558/078)

Min Raj Panday (28560/078)

ABSTRACT

DeNoisify : Intelligent audio denoising system presents an advanced noise removing system. The system is build by integrating the Wave U-Net deep learning model with MMSE-STSA post-processing. The model is designed to effectively isolate the noise from the noisy audio signals. Initially, the Wave-U-Net architecture is employed to process the input audio, extracting multi-scale features and reduce the noise in the audio signals. The resulting output is then refined using MMSE-STSA, which enhances the audio by leveraging the statistical independence of audio components. To integrate this system, we have used Django technology, enabling a seamless framework for processing and delivering the denoised audio. This hybrid approach demonstrates significant potential in improving noise reduction and audio clarity. This project offers a robust solution for audio enhancement applications. However, limitations include the system's dependency on high-quality input data, potential performance degradation with diverse noise types, and computational resource demands, necessitating further optimization and testing.

Keywords: *Audio Denoising, Deep Learning, Django, Speech Enhancement, Wave-U-Net, MMSE-STSA*

TABLE OF CONTENTS

TITLE PAGE	ii
SUPERVISOR'S RECOMMENDATION	iii
CERTIFICATE OF APPROVAL	iv
ACKNOWLEDGEMENT.....	v
ABSTRACT.....	vi
LIST OF ABBREVIATIONS	ix
LIST OF FIGURES	x
LIST OF TABLES	xi
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives.....	2
1.4 Scope and Limitation.....	2
1.4.1 Scope	2
1.4.2 Limitation	3
1.5 Development Methodology	3
1.6 Report Organization	4
CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW	6
2.1 Background Study	6
2.2 Literature Review	7
CHAPTER 3 SYSTEM ANALYSIS	9
3.1 Project Analysis.....	9
3.1.1 Requirement Analysis	9
3.1.2 Feasibility Study.....	12
3.1.3 Analysis	15
CHAPTER 4 SYSTEM DESIGN	18
4.1 Design.....	18
4.1.1 Refinement of Class Diagram	20
4.1.2 Refinement of Sequence Diagram.....	21
4.1.3 Refinement of Activity Diagram	22
4.1.4 Component Diagram	23

4.1.5 Deployment Diagram	24
4.2 Algorithm Details	25
4.2.1 Convolutional Neural Network	25
4.2.2 Minimum Mean Square Error Short -Time Spectral Amplitude.....	27
CHAPTER 5 IMPLEMENTATION AND TESTING	30
5.1 Implementation.....	30
5.1.1 Tools Used.....	30
5.1.2 Implementation details of Modules	32
5.2 Testing.....	37
5.2.1 Test Case for Unit Testing.....	37
5.2.2 Test Case for System Testing.....	40
5.3 Result Analysis.....	41
5.3.1 Performance Metrics	42
CHAPTER 6 CONCLUSION AND FUTURE RECOMMENDATIONS.....	44
6.1 Conclusion.....	44
6.2 Future Recommendation	44
REFERENCES.....	46
APPENDICES	

LIST OF ABBREVIATIONS

AI:	Artificial Intelligence
API:	Application Programming Interface
CNN:	Convolutional Neural Network
CPU:	Central Processing Unit
CSS:	Cascading Stylesheets
dB:	Decibel
GPU:	Graphical Processing Unit
HTML:	Hypertext Markup Language
Hz:	Hertz
ISTFT:	Inverse Short-Time Fourier Transform
JS:	JavaScript
MMSE-STSA:	Minimum Mean Square Error Short-Time Spectral Amplitude
PESQ:	Perceptual Evaluation of Speech Quality
RAM:	Random Access Memory
ReLU:	Rectified Linear Unit
SNR:	Signal-to-Noise Ratio
STFT:	Short-Time Fourier Transform
STOI:	Short-Time Objective Intelligibility
UI:	User Interface

LIST OF FIGURES

Figure 1.1: Incremental Model	3
Figure 3.1: Use Case Diagram of Intelligent Audio Denoising System	10
Figure 3.2: Gantt Chart	14
Figure 3.3: Class Diagram for Intelligent Audio Denoising System	15
Figure 3.4: Sequence Diagram for Intelligent Audio Denoising System	16
Figure 3.5: Activity Diagram for Intelligent Audio Denoising System	17
Figure 4.1: High-Level System Design of DeNoisify	18
Figure 4.2: Refined Class Diagram for Intelligent Audio Denoising System	20
Figure 4.3: Refined Sequence Diagram for Intelligent Audio Denoising System	21
Figure 4.4: Refined Activity Diagram of Intelligent Audio Denoising System	22
Figure 4.5: Component Diagram of Intelligent Audio Denoising System	23
Figure 4.6: Deployment Diagram of Intelligent Audio Denoising System	24
Figure 4.7: Architecture of Wave-U-Net	25
Figure 4.8: Block Diagram of MMSE-STSA	27
Figure: 5.1 Implementation of Wave-U-Net Module	32
Figure 5.2: Implementation of MMSE-STSA	34
Figure 5.3: Training Loop of Wave-U-Net	36
Figure 5.4: Accuracy and Loss Curve for Wave-U-Net	42

LIST OF TABLES

Table 3.1: Use-case Scenario of Intelligent Audio Denoising System.....	11
Table 3.2: Schedule Feasibility.....	13
Table 5.1: Test Case for User Log In.....	37
Table 5.2: Test Cases for Audio Input	38
Table 3.3: Test Cases for Audio Processing	38
Table 5.4: Test Cases for Download Functionality	39
Table 5.5: Test Case for DeNoisify System.....	40
Table 5.6: Analysis Table for Wave-U-Net.....	41
Table 5.7: Summery of Performance Metrics.....	43

CHAPTER 1

INTRODUCTION

1.1 Introduction

Today's fast-paced digital world is demanding more and more clear audio, not just for content creators like YouTubers and musicians, but everywhere that people are communicating, such as meetings, podcasting, for hearing aids and real-time communication. However, it remains a tough challenge to remove unwanted noise in audio recordings, in particular under uncontrolled conditions. Whether it be wind, traffic, fan noise or crowd noise, background sound has the potential to greatly diminish clarity, serve as a nuisance to the listener or even render the use of audio data unfeasible in downstream systems such as voice recognition.

Our approach seeks to address this shortcoming by integrating the Wave-U-Net, a CNN adapted for end-to-end source separation and the MMSE-STSA (Minimum Mean Square Error Short-Time Spectral Amplitude estimator), a traditional algorithm for speech enhancement. Through a combination of the two phantom methods the system aims at reducing background noise while maintaining naturalness and intelligibility of speech.

The difference to traditional denoising methods is that many of such techniques do modify the original audio or do not work well in complex noise scenarios, while our method uses the learning capability of CNNs, combined with the fact that statistical signal processing is a trustful mechanism. The result is a system which is a hybrid which works well in a range of real-world noise conditions and could be used as a standalone desktop tool or as a plugin for a real-time audio application.

1.2 Problem Statement

We are surrounded by noise wherever we go. Noise is present, for instance, in the street (e.g., cars passing by, street construction work), in the car (e.g., engine noise, wind), the office (e.g., PC fan noise, air dusts), the restaurant (e.g., people talking at nearby tables), and the department stores (e.g., telephone ringing, sales representatives talking). As these examples illustrate, noise appears in different shapes and forms in daily life.

Noise can be stationary, that is, does not change over time, such as the fan noise coming from PCs. Noise can also be non-stationary, such as noise during an event with several celebrity speakers. While capturing the audio, we encountered significant background noise including audiences noise, fan sounds, and lens zooming noise. These disturbances compromised the clarity of speakers' voices, reflecting the need for an intelligent noise reduction system. Clearly, the task of suppressing non-stationary noise is more difficult than that of stationary noise.

DeNoisify aims to tackle these issues by developing the system that uses the Convolution Neural Network (CNN) to distinguish between unwanted ambient sounds and meaningful speech. This web based application does not just remove noise but also integrate in many important real-world applications, including mobile speech communication, hearing aids designs and robust speech recognition. The goal of this system is to improve the intelligibility and quality of a noisy speech signal degraded in adverse conditions.

1.3 Objectives

The main objectives of this project are:

1. To develop an audio denoising system that combines Wave-U-Net and MMSE -STSA for effective noise removal.
2. To enhance speech signals while preserving speech quality and minimizing artifacts.
3. To provide a user-friendly interface for uploading and processing audio files.

1.4 Scope and Limitation

1.4.1 Scope

1. The system is made to work with WAV audio files, either mono or stereo.
2. It can be used in areas such as online conferencing, podcasts, interviews, and assistive hearing devices.
3. It provides balanced performances across a variety of noise types by combining deep learning and signal processing.

1.4.2 Limitation

1. The current model is trained primarily on English speech datasets and may not generalized perfectly across all languages or dialects.
2. Real-time performance is limited by hardware constraints, especially when using the deep learning modules on lower-end systems.
3. The model does not perform well on music signals or non-speech audio.

1.5 Development Methodology

The project is developed using the Incremental Model. In the incremental model, a complex project is divided into smaller, self-contained modules or increments. Each increment represents a partial system with added functionality, allowing for the project's gradual development. The key feature of this model is its iterative process, where increments are developed, tested, and integrated one after another into the evolving system.

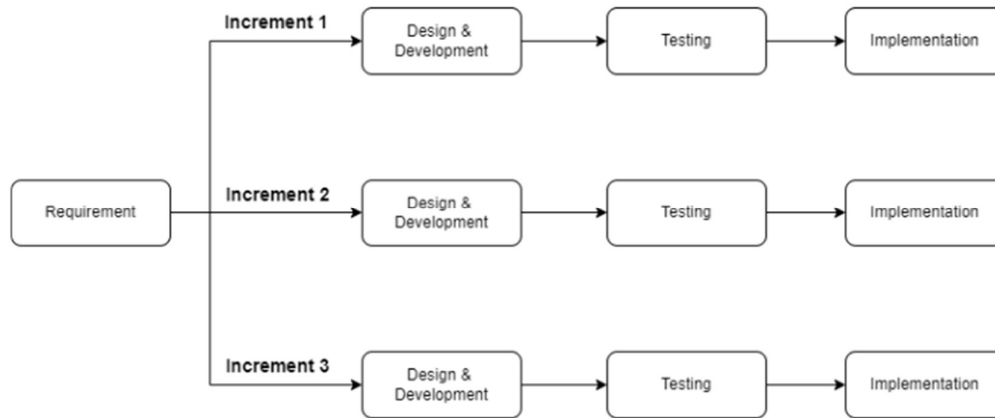


Figure 1.1: Incremental Model

1. Iteration 1:

In the first iteration, we implemented traditional speech enhancement technique like MMSE-STSA. This initial version provided a simple but functional noise reduction framework. The performance was limited, often introducing artifacts like musical noise in complex environments.

2. Iteration 2:

In the second iteration, we deployed and trained the Wave-U-Net model using the paired noisy and clean samples from LibriSpeech and UrbanSound8k datasets. This model significantly improved the quality of the denoised audio compared to the traditional approach. However, some residual noise and distortions were still observed.

3. Iteration 3:

In the third iteration, we combined the strengths of both approaches. We developed a hybrid pipeline by integrating the MMSE-STSA as a post-processing step with the Wave-U-Net output. In order to make it user friendly, we developed a simple web interface using the Django. This allows the users to upload noisy audio and download denoised results.

1.6 Report Organization

This report of “DeNoisify: Intelligent Audio Denoising System” is separated into 6 different chapters.

Chapter 1: This chapter introduces the project along with its objectives, scope, limitations and the development methodology used to develop the project.

Chapter 2: This chapter discusses the background relating to the project and the summary of past research papers and reports related to the current project.

Chapter 3: This chapter includes the analysis of the requirements of the project which contains use case diagrams and scenarios, class diagrams, activity diagrams and sequence diagrams along with Gantt chart to visualize the project schedule.

Chapter 4: This chapter includes system design and UML diagrams of the system such as refined class diagrams, refined sequence diagrams and refined activity diagram.

Chapter 5: This chapter describes about the tools used in the development of this project. It also contains details of testing done and the result analysis of the system.

Chapter 6: This chapter contains the conclusion and the possible future recommendations for this project.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The performance of voice communication systems deteriorates significantly in challenging acoustic environments. For instance, a user inside a noisy vehicle may become difficult to comprehend due to interference from noise sources such as the engine, wind, ventilation system, and road conditions, leading to reduced speech quality and diminished intelligibility. The DeNoisify system is designed to enhance the performance of voice communication by improving the quality and intelligibility of corrupted speech.

Traditional audio denoising models often focus on the magnitude spectrum, neglecting phase information and making their performance dependent on hyper-parameters for the spectral front-end. To overcome this, the Denoisify system employs an end-to-end approach in the time-domain, allowing the modeling of phase information and avoiding fixed spectral transformations. Given the high sampling rates of audio, incorporating a long temporal input context at the sample level is challenging yet crucial for achieving high-quality denoising due to long-range temporal correlations.

DeNoisify utilizes the Wave-U-Net, and adaptation of the U-Net designed for the one-dimensional time domain, which resamples feature maps to compute and integrate feature across various time scales. Following the Wave-U-Net processing, the Minimum Mean-Square Error Short-Time Spectral Amplitude (MMSE-STSA) technique is applied as a post-processing step. MMSE-STSA enhances the denoised output by estimating the clean speech signal's amplitude spectrum, minimizing residual noise while preserving the speech's natural characteristics. This method leverages statistical properties of the noise and speech signals to refine the results, ensuring improved intelligibility and quality in the final denoised audio output.

2.2 Literature Review

DeNoisify is an AI-powered denoising system that can help to remove the background noise that harms the quality and clarity of the communicated speech and generate the noise free sound to improve the better understanding of the speech.

This project is based on hybrid model which is design and implementation by combining deep learning techniques i.e. U-Net architecture with the most popular statistical based minimum mean-square error short-time spectral amplitude (MMSE-STSA) estimator for speech enhancement.

Recently, with the development of deep learning, speech enhancement has made major breakthroughs in deep learning-based methods. These methods include convolutional neural network (CNN), recurrent neural network (RNN), etc. Among them, speech enhancement based on convolutional U-Net has also attracted much attention. [1] UNet, originally developed for biomedical image segmentation [2], has been effectively adapted for speech denoising. Its encoder-decoder structure with skip connections allows it to capture both local and global features of audio signals. Wave-U-Net, a 1D variant of U-Net for raw audio signals, has shown remarkable performance in source separation and denoising tasks [3].

The Wave-U-Net architecture, adapted by Macartney and Weyde (2018) for speech denoising, exhibits superior performance on the VoiceBank-DEMAND dataset. This model processes raw audio directly preserving phase and temporal continuity achieving landmark improvements in perceptual metrics. The study reported PESQ scores around 2.4 and STOI values of 0.83, outpacing traditional CNN or CRN baselines [4]. Its lightweight design (reduced depth compared to music-separation variants) makes it particularly well-suited for real-time or resource-constrained environments while maintaining competitive quality.

The MMSE-STSA (Minimum Mean Square Error – Short-Time Spectral Amplitude) estimator, originally introduced by Ephraim and Malah in 1985 [5], remains a cornerstone in statistical speech enhancement due to its low-complexity and high efficacy in noise suppression. This method has been further evaluated in practical implementations like Simit Shah and Roma Patel’s 2015 study titled “MMSE-STSA Based Techniques for Single Channel Speech Enhancement Application” [6]. Their research demonstrates how MMSE-STSA can be effectively tailored for monaural speech enhancement systems, reinforcing the method’s reliability in real-time applications.

Combining Wave-U-Net with MMSE-STSA yields a hybrid enhancement pipeline, blending deep learning’s ability to suppress broad noise patterns with the statistical precision of post-filtering. Experimental results indicate that this combination increases PESQ to around 2.6–2.7 and STOI to 0.84–0.85, outperforming standalone Wave-U-Net while approaching more complex spectrum-domain models . Notably, DenseCRN-based models and transformer-based systems like CMGAN may achieve PESQ 2.8 or STOI 0.94, but at the cost of significantly larger model size and computational burden [7] .

CHAPTER 3

SYSTEM ANALYSIS

3.1 Project Analysis

3.1.1 Requirement Analysis

The system must have the following functional and non-functional requirements.

i. Functional Requirements

The requirements outlined here are essential for the system's operation. They primarily focuses on the functionalities that the system should provide. These include:

1. Audio Upload:

- The system must allow the users to upload noisy audio file.
- The system should validate file to ensure supported formats and size limits

2. Noise Reduction:

- The system must process the uploaded noisy audio through the Wave-U-Net model
- Post-processing using MMSE-STSA should refine the enhanced output

3. Audio Playback:

- Users should be able to play back the original noisy audio and the denoised audio.

4. User Authentication:

- The system should allow users to log in or sign up to download and view audio processing history.

5. Download Functionality:

- The system must allow users to download the denoised audio file if user is logged in.

6. Administrative Functions

- Administrators should be able to manage user accounts, monitor uploaded and processed files.

- **Use Case Diagram**

The figure 3.1 shows the use case diagram of the Intelligent Audio Denoising System, the user uploads the audio file containing noisy speech which then the system processes to give the denoised audio file.

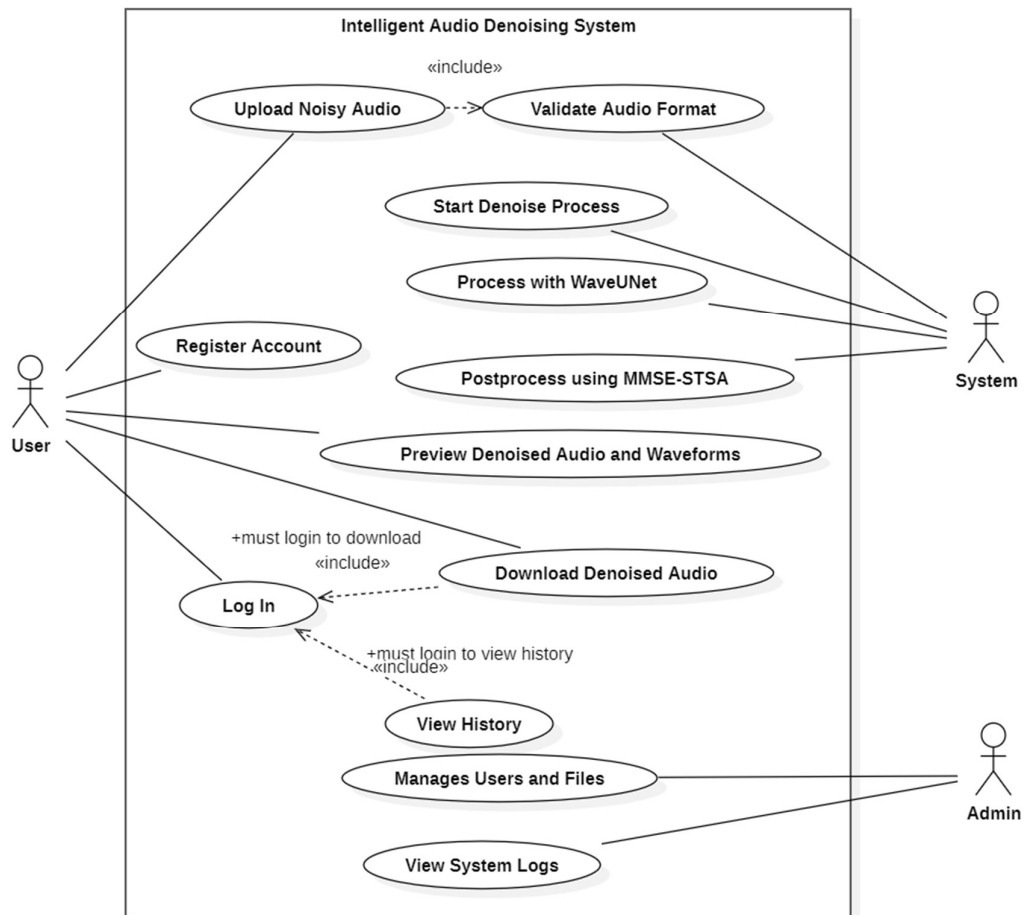


Figure 3.1: Use case diagram of Intelligent Audio Denoising System

Table 3.1: Use-case scenario of Intelligent Audio Denoising System

Use case: Audio Denoising with DeNoisify	
Primary Actor:	User
Secondary Actor:	System,Admin
Flow of event:	i. User navigates to the DeNoisify web page. ii. User clicks Upload Audio() and selects a noisy audio file from the device. iii. System validates the file format and size. iv. User clicks Denoise() to start the process. v. System processes the audio using the Wave-U-Net model() to reduce noise. vi. The output from the Wave-U-Net is further refined using MMSE-STSA() postprocessing. vii. System displays a processing progress bar(). viii. Once processing is complete, the system generates a download link(). For the denoised audio. ix. User clicks Download() to save the cleaned audio file.
Pre-condition:	The system must be in running state. The User must login to download.
Post-condition:	The user must receive a denoised audio file. The user must be able to view the history.
Quality:	The result must be correct.

Non-Functional Requirements

- **Usability:** The system will be user-friendly and easy to navigate. It will feature clearly labeled buttons, straightforward options and simple instructions to ensure a smooth user experience for all users.
- **Reliability:** The system will be dependable and consistent, operating without unexpected interruptions or failures.
- **Performance:** The system should be able to process 10-second audio clips in a matter of seconds.

3.1.2 Feasibility Study

i. Technical Feasibility

The technical feasibility involves the evaluation of the hardware, software and other technology requirements for the construction of the project.

- **Hardware Requirements**

The project was run on a local machine with 6 GB of GPU with a processor of Intel Core I5 13th gen with 8 GB of RAM. The training required 97% of memory of the GPU, 95% of CPU and 91% of total RAM. So to train the model it is recommended to use at least a hardware system that can match the GPU capability as mentioned above.

- **Software Requirements**

The software requirements for this project involves using the language Python with Django as web framework to manage user interactions, file uploads and inference requests, along with the machine learning libraries like Pytorch to implement and train the Wave-U-Net model. Similarly, the specific libraries such as NumPy and Librosa are used for the audio preprocessing. The project also requires the Pandas and Matplotlib for visualization. HTML, CSS, Bootstrap and JavaScript are used for the project's frontend while Django is utilized for the backend.

ii. Operational Feasibility

The Intelligent Audio Denoising System operates by allowing users to upload audio file containing the noise signals. The system processes the input audio and then provides the clean audio as output. This ensures the project's operational feasibility, making it an inclusive and effective communication tool.

iii. Economic Feasibility

This analysis accounts for all the key expenses related to the final development of the audio denoising system like potential additional costs for enhancing RAM for the training of the Neural Network. The system is designed to efficiently process noisy audio inputs and provide clean audio all while staying within the existing budget. As no further purchases are required for deployment, the project is cost-effective and economically feasible.

iv. Schedule Feasibility

We have used a Gantt chart to highlight the time period for the project.

Table 3.2: Schedule Feasibility

Name	Start Date	End Date	Duration
Planning and Topic Selection	Thu 5/1/25	Wed 5/7/25	5 days
Literature Review	Fri 5/9/25	Fri 5/23/25	11 days
Data Collection	Mon 5/26/25	Fri 6/13/25	15 days
Wave-U-Net Implementation	Mon 6/16/25	Fri 7/11/25	20 days
MMSE-STSA Implementation	Mon 7/14/25	Wed 7/23/25	8 days
Integration and Testing	Thu 7/24/25	Tue 8/5/25	9 days
UI Development	Wed 8/6/25	Fri 8/15/25	8 days
System Evaluation	Mon 8/11/25	Wed 8/20/25	8 days
Documentation	Tue 5/1/25	Sun 8/24/25	83 days

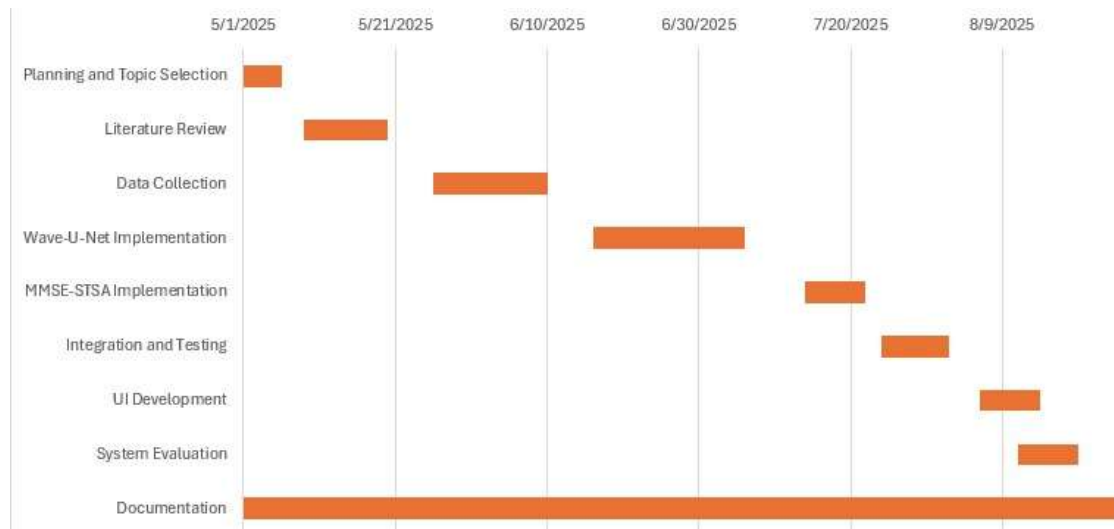


Figure 3.2: Gantt Chart

3.1.3 Analysis

- **Object modelling using Class and Object Diagrams**

The class diagram depicts a static view of an application. It is used to show the classes in a system and the associations between these classes.

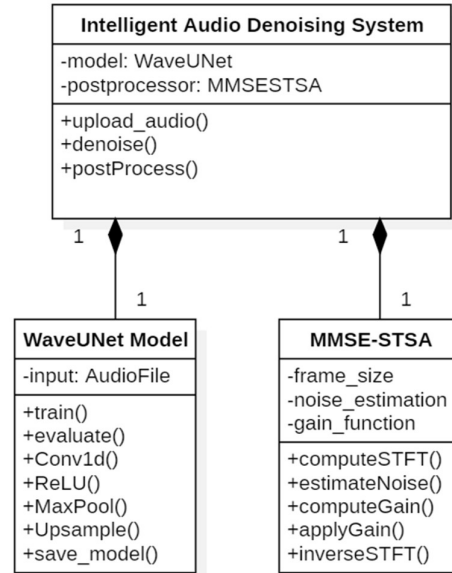


Figure 3.3: Class Diagram for Intelligent Audio Denoising System

In figure 3.3, there are 3 main classes: Intelligent Audio Denoising System, WaveUNet model and MMSE-STSA. The Intelligent Audio Denoising System class contains WaveUNet Model and MMSE-STSA postprocessor attributes and methods for audio upload, audio analysis , denoise and audio post process. The Wave-U-Net model represents the Convolutional Neural Network architecture used for the audio denoising. It includes attributed for encoder layers, bottleneck, decoder layers and skip connections , along with methods for convolution, ReLu activation, max pooling and upsampling. The Wave-U-Net is an important component of the system for transforming noisy speech into cleaner speech. The MMSE-STSA represents the statistical signal processing algorithm used for the post-processing. It includes attributes for frame size, noise estimation and gain function, along with methods for STFT computation, noise estimation, gain calculation and inverse STFT.

- **Dynamic modelling using State and Sequence Diagrams**

The sequence diagram is used primarily to show the interactions between objects in the sequential order that those interactions occur. It provides a dynamic perspective on the system's behaviour over time.

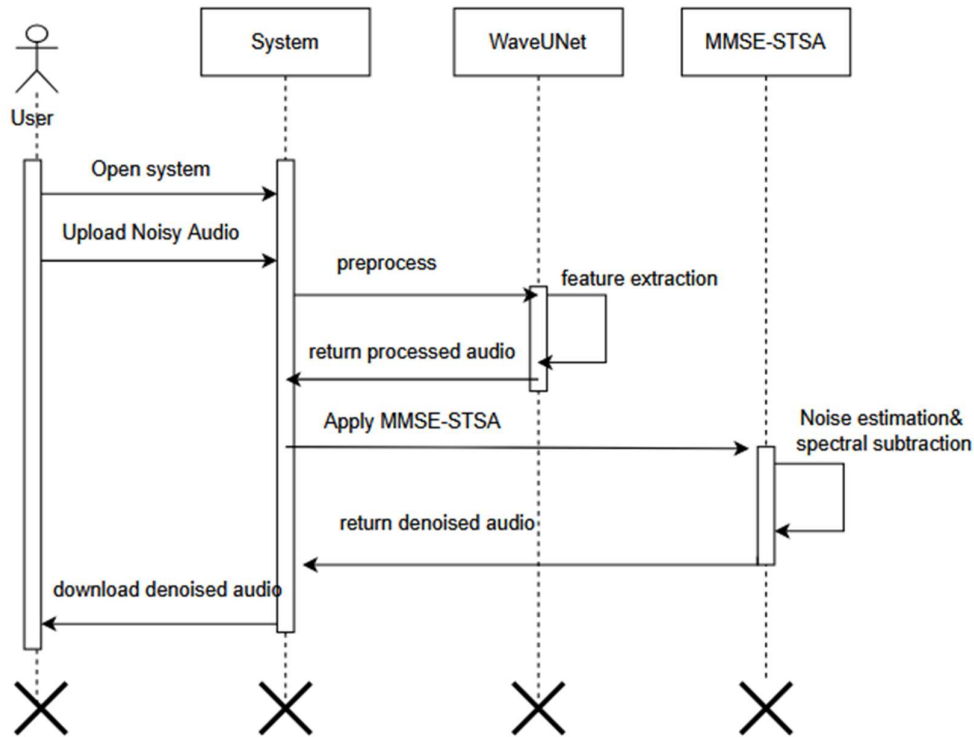


Figure 3.4: Sequence Diagram for Intelligent Audio Denoising System

The sequence diagram in the Figure 3.4 illustrates the interaction between the user, the audio denoising system, WaveUnet model and MMSE-STSA. Users can initiate interaction by opening the system and uploads a noisy audio file. The system forwards this audio to the model, which extracts the important speech and returns the processed audio to the system. Then, the system applies the MMSE-STSA algorithm as a post-processin step to further enhance the audio. Finally, a denoised audio output is provided to user for visualization, playback and download .

- **Process modelling using Activity Diagrams**

An Activity diagram is a form of UML diagram that shows the flow of control or a sequence of events in a system visually. It displays step-by-step work processes that allows for choice, iterations and concurrency.

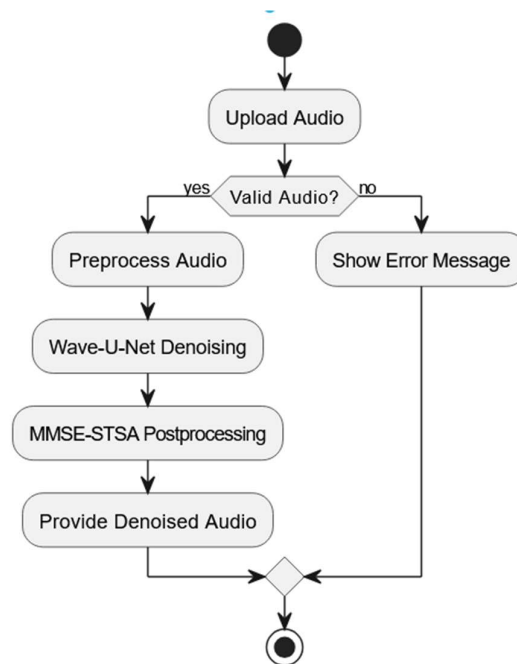


Figure 3.5: Activity Diagram for Intelligent Audio Denoising System

In the activity diagram in Figure 3.5, the user initiates the process by opening the system and providing input audio, which can be in the WAV or MP3 format. When the audio is valid, it is preprocessed to prepare it to be denoised. The audio is then preprocessed with the Wave-U-Net model to eliminate a majority of the noise and then processed through the MMSE-STSA algorithm to further refine the audio by eliminating any remaining noise. Lastly, the user is presented with the denoised audio. When the audio is invalid, the system will immediately give an error message.

CHAPTER 4

SYSTEM DESIGN

4.1 Design

System design is the process of designing the architecture and components of a software system to meet specific requirements. This involves converting user requirements into a detailed blueprint that guides the implementation phase.

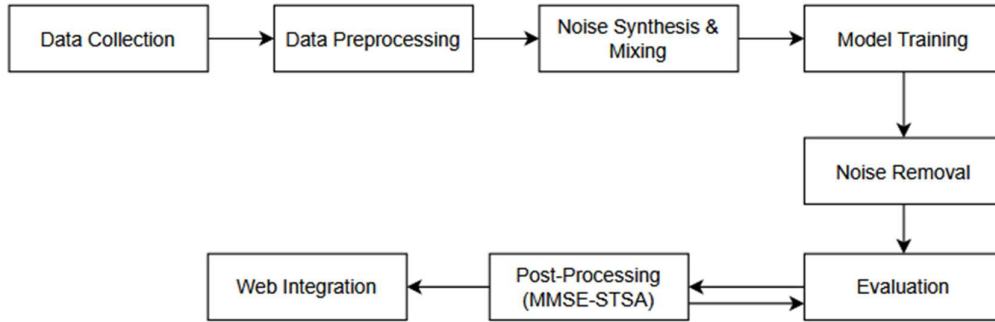


Figure 4.1: High-Level System Design of DeNoisify

The system architecture is shown in Figure 4.1. The procedures involved are as:

- **Data Collection**

For the system, we used two datasets like Librispeech (for clean speech) and UrbanSound8k (for environmental noise). The dataset was collected from the Kaggle.

- **Data Preprocessing**

All audio files were converted to WAV format with a 16 kHz sampling rate. Volume levels were normalized, silence at the start and end is trimmed, and recordings were split into 3–5 second segments for easier processing. Spectrograms were created using STFT with a window size of 512 samples and hop length of 256. Minor augmentations like $\pm 5\%$ speed or $\pm 2\%$ pitch adjustments were applied to increase robustness.

- **Noise Synthesis and Mixing**

A random noise signal was selected and blended with the clean audio signal to create a simulated noisy audio signal. This mixture was used as the input for the model training.

- **Model Training**

For the model training, we used Wave-U-Net architecture. The model was trained using pairs of noisy and clean audio. We used a batch size of 8 and 16, learning rate of 0.0001 and run for around 50 epochs. During training, the model gradually adjusts its internal parameters to minimize the difference between its output and the actual clean audio.

- **Evaluation**

The performance of the model was evaluated by observing the training loss and validation loss values. Mean squared error(mse) was used as loss function. The noise removed speech signals obtained were analyzed and evaluated by careful listening. After completing the training, the trained model was loaded and the test dataset was then loaded. The test dataset was processed through the Wave-U-Net model, generating predictions in the form of denoised audio waveforms for each noisy input in the evaluation set. The predicted denoised audio was then compared against the ground truth clean audio using objective metrics such as PESQ, STOI, and SNR, to assess the performance and quality of the noise removal process.

- **Post-Processing**

Once the initial denoised audio is generated by the Wave-U-Net model, it is refined in post-processing. This is done by performing operations such as MMSE-STSA noise reduction and smoothing, in order to improve the final audio quality.

- **Web Integration**

A web application has been developed using Django. Within this app, an audio upload component is built, which allows users to provide their noisy audio files. Then, the system validate the input and process it using the Wave-U-Net model. Once the model processes the input, it generates the predicted denoised audio, which is then refined using post-processing techniques i.e MMSE-STSA noise reduction. Finally, the denoised audio is returned to the user and served via the application interface.

4.1.1 Refinement of Class Diagram

The refined class diagram is used to show the detailed view of all the classes in a system and the associations between these classes.

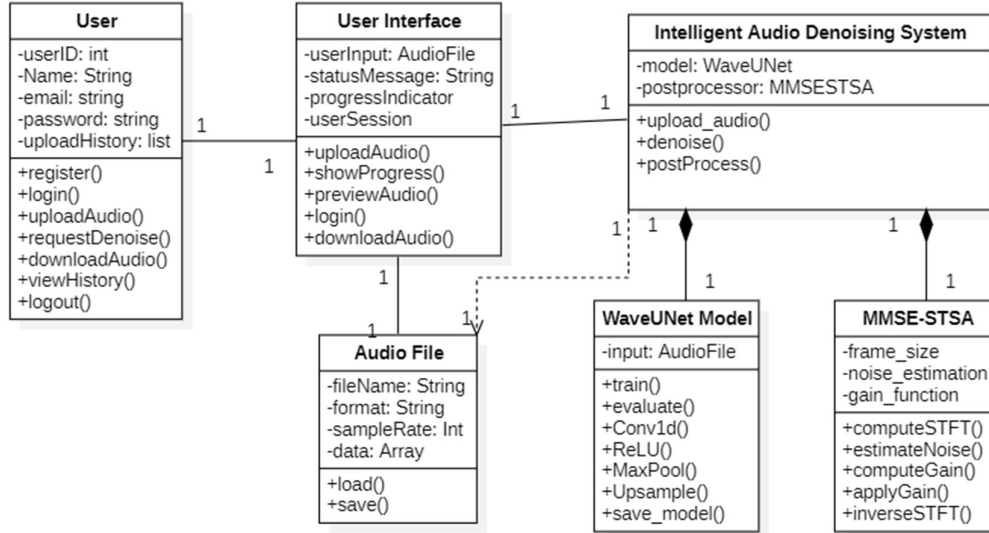


Figure 4.2: Refined Class Diagram for Intelligent Audio Denoising System

Figure 4.2 shows the refined class diagram. There are 6 main classes: Intelligent Audio Denoising System, WaveUNet Model, MMSE-STSA, User, User Interface, Audio File. The User class allows users to register or login (if already registered), upload the audio file, download the denoised audio and view the history. It interacts with the User Interface that accepts user input and displays messages. The File contains the properties like file name, format, sample rate and data. It is passed to the Intelligent Audio Denoising System class as input to detect the useful speech and get the denoised audio file as output.

The Intelligent Audio Denoising System centralizes the operation, incorporating WaveUNet for audio denoising and MMSE-STSA for further enhancement. It processes noisy audio input, reduces background noises and reconstructs clean audio output ensuring seamless integration of all components.

4.1.2 Refinement of Sequence Diagram

A refined sequence diagram illustrates the interactions between different objects or components in a system over time. It provides a detailed view of the chronological order of messages exchanged among these objects.

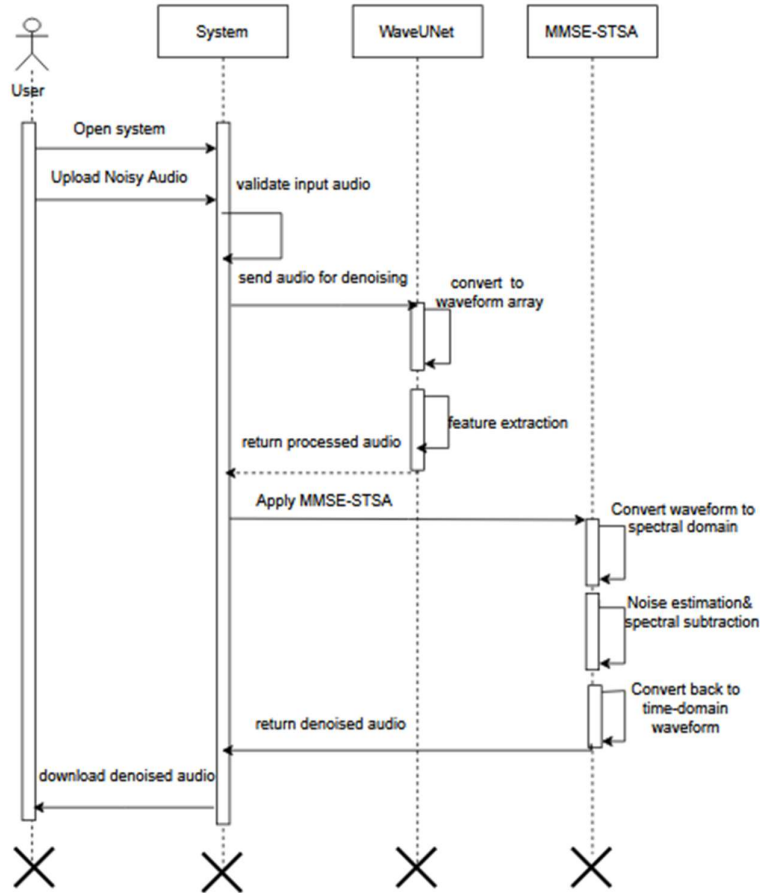


Figure 4.3: Refined Sequence Diagram for Intelligent Audio Denoising System

Figure 4.3 shows the refined sequence diagram for Sign Input. The user initially visits the system web and uploads the noisy audio file as input; the system then checks where the input file is valid or not and sends the valid input audio file to WaveUNet.

The WaveUNet, first converts the audio file into a numerical waveform array and then proceeds through the encoding layers, where convolution and ReLU activations extract features and downsample, then a bottleneck layer extracts high-level audio representations. The decoding layers are used to build the audio through upsampling and skip connections to reduce any chance of losing finer details and give an audio that is partially denoised.

This partially denoised output is next forwarded to the MMSE-STSA processor which first transforms the waveform into the spectral domain by Short-Time Fourier Transform (STFT). MMSE-STSA estimates the power spectrum of noise and uses spectral subtraction to cut additional noise. Lastly, the improved audio is transformed back to the time-domain waveform and sent back to the system. The denoised audio is then presented to the user

4.1.3 Refinement of Activity Diagram

Refined activity diagram provides the detailed and comprehensive representation of the flow of the flow of control and sequence of events in a system.

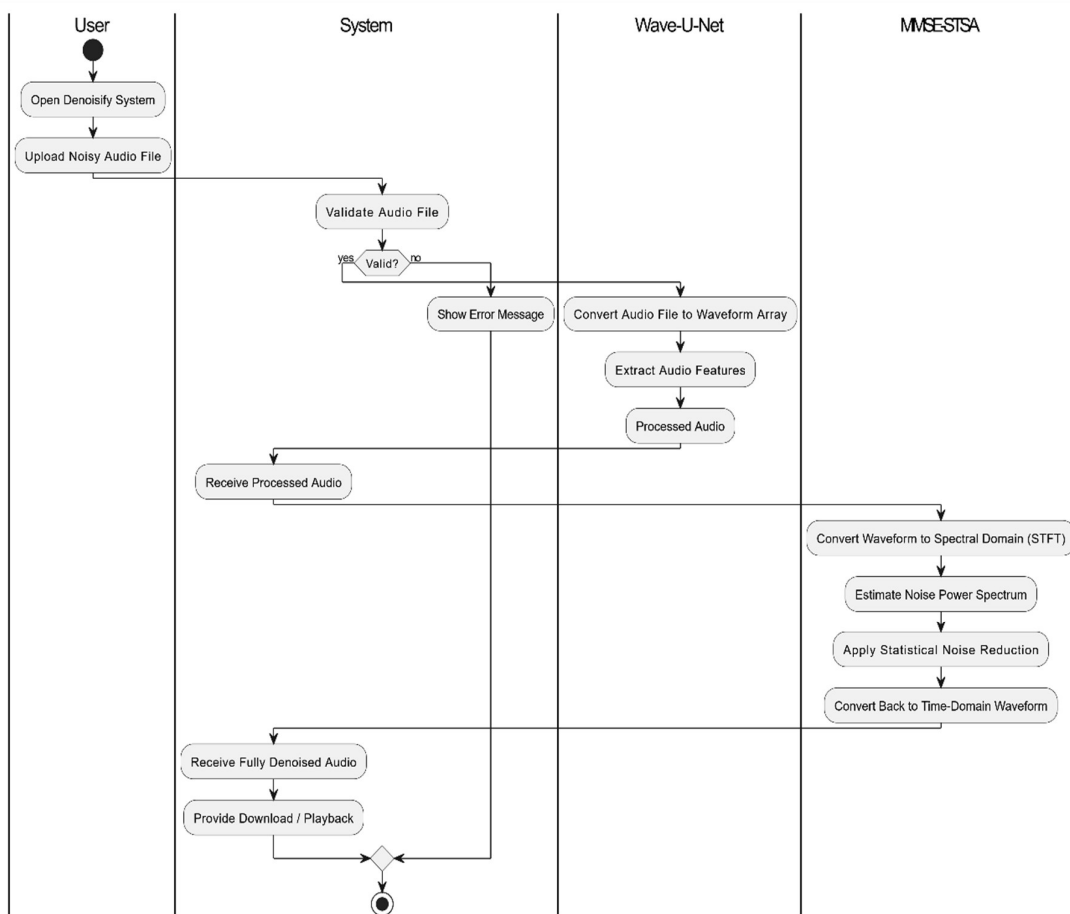


Figure 4.4: Refined Activity Diagram of Intelligent Audio Denoising System

The refined activity diagram in Figure 4.4 shows the activity diagram for audio denoising. The process begins with the user opening the Denoisify system and uploading a noisy audio file. The system first validates the audio to ensure it is in a supported format. Once validated, the audio is processed by the Wave-U-Net model, which converts the audio into a waveform array, extracts relevant audio features, and performs initial denoising. The partially denoised audio is then passed to the MMSE-STSA module, where it is transformed into the spectral domain, noise is estimated, and statistical noise reduction is applied. The enhanced audio is then converted back to the time-domain waveform and provided to the system. Finally, the user is able to listen to the denoised audio or download.

4.1.4 Component Diagram

A component diagram divides the system into distinct functional units. Each component has a specific task or function and communicates only with other crucial components.

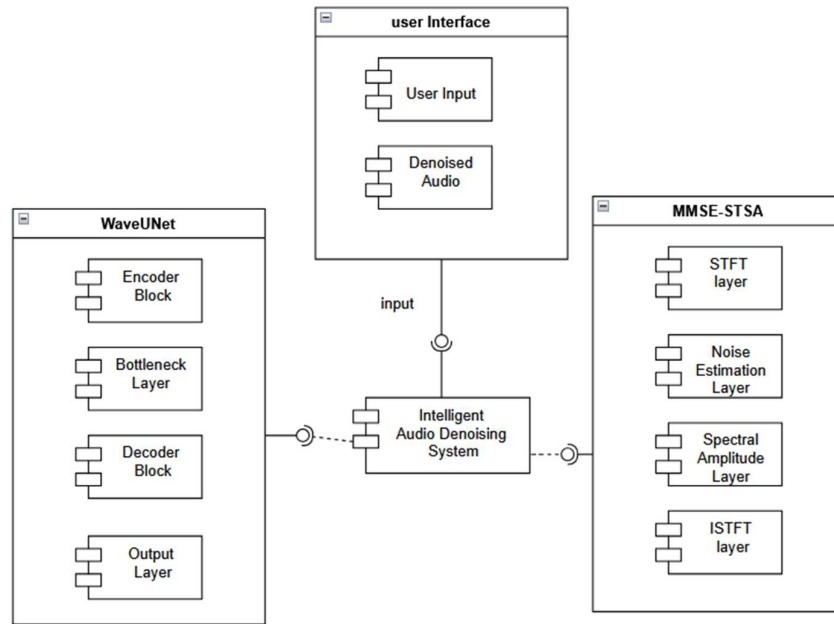


Figure 4.5: Component Diagram of Intelligent Audio Denoising System

The component diagram in Figure 4.5 shows the component diagram for the Denoisify system. This project integrates Wave-U-Net and MMSE-STSA modules for audio denoising. The Wave-U-Net module converts uploaded audio files into waveform arrays, extracts audio features, and performs initial denoising. The MMSE-STSA module then transforms the partially processed audio into the spectral domain, estimates the noise, applies statistical noise reduction, and converts the audio back to the time-domain waveform. The User Interface

accepts noisy audio input from the user and provides the fully denoised audio for playback or download. These components are connected to the central “Intelligent Audio Denoising System” component, indicating their integration in this system.

4.1.5 Deployment Diagram

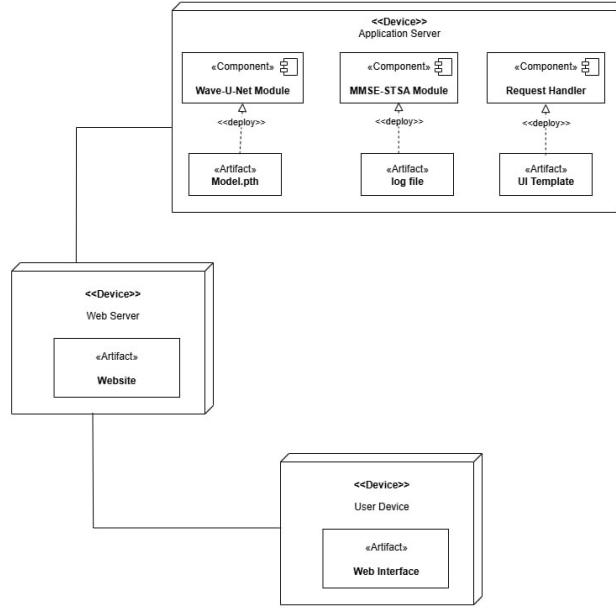


Figure 4.6: Deployment Diagram of Intelligent Audio Denoising System

The deployment diagram of Denoisify illustrates how the system is distributed across devices like User Device, Web server and Application Server. At the front, users interact with the system through their devices, uploading noisy audio files and receiving the cleaned output via a web interface. These requests first reach the web server, which acts as a bridge between the user and the backend, forwarding audio files to the application server for processing. The application server is the heart of the system, where the Wave-U-Net module predicts the denoised audio, the MMSE-STSA module performs additional post-processing to remove residual noise and maintains log files for tracking. Once processed, the refined audio travels back through the web server to the user, ensuring a smooth and efficient pipeline from upload to output.

4.2 Algorithm Details

The project implements the following algorithms:

4.2.1 Convolutional Neural Network

WaveUNet is a specialized neural network designed for audio processing tasks, such as denoising, adapting the principles of the U-Net architecture to one-dimensional time-domain signals. This algorithm learns audio features using convolutional layers and updates weights based on corresponding audio labels, enabling the trained network to separate or denoise unlabeled audio signals.

Wave UNet primarily consists of three key layers: Convolution layer, upsampling layer, and a different output layer, with additional components like skip connections and learned upsampling for enhanced performance.

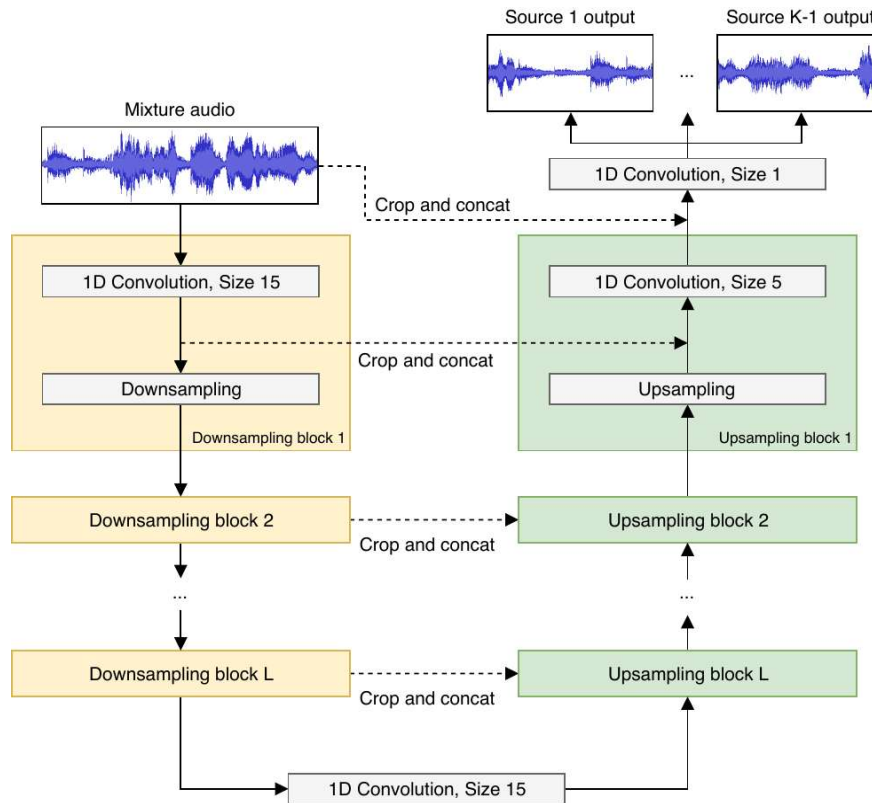


Figure 4.7: Architecture of Wave-U-Net

- **Convolution Layer**

The convolution layer is the core, carrying the majority of the network's computational load. It performs a dot product between a kernel (or filter) and the input audio segments to generate an activation map, where the kernel slides over the entire audio signal.

For a kernel K of size M (number of samples) and depth D (number of channels) applied to an input patch I of the same size, the dot product is computed as follows:

$$\text{Dot product} = \sum_{i=1}^M \sum_{j=1}^M \sum_{k=1}^D K[i, j, k] \cdot I[i, j, k] \quad \dots (1)$$

Where $K[i, j, k]$ is the value of the kernel at depth k and position (i, j) , $I[i, j, k]$ is the value of the input patch at depth k and position (i, j) , and M is the kernel size. This results in a single value placed in the output feature map, repeated as the kernel slides across the input audio. The output size W_{out} for input size W , kernel size M , padding P , and stride S is calculated as:

$$W_{out} = \frac{W - M + 2P}{S} + 1 \quad \dots (2)$$

This gives us an output volume size of $W_{out} \times W_{out} \times D_{out}$ where D_{out} is the number of kernels, capturing multi-scale audio features.

- **Upsampling Layer**

After the downsampling path, the upsampling layer increases the spatial resolution of the feature map, effectively reconstructing the audio signal. It reduces the temporal compression introduced by downsampling, preserving detailed audio characteristics. WaveUNet uses learned upsampling, computing interpolated features as a weighted average between consecutive samples. The output size after upsampling is adjusted based on the downsampling factor, typically doubling the resolution, and is combined with skip connections from the convolution layer for feature integration. The output size can be approximated as:

$$W_{out} = \frac{W \cdot 2 - M}{S} + 1 \quad \dots (3)$$

Where W is the input size after downsampling, M is the upsampling kernel size, and S is the stride, through the exact formulation depends on the learned weights.

- **Different Output Layer**

In the final layer, every neuron is connected to the previous layers, allowing the combination of multi-scale features detected earlier. WaveUNet predicts $K-1$ denoised audio sources, and the last source is computed as the difference from the input mixture M , enforcing the constraint $S_K = M - \sum_{j=1}^{K-1} \hat{S}_j$. The output is used for generating the denoised audio signal, facilitating decisions on audio quality.

- **ReLU activation Layer**

The Rectified Linear Unit (ReLU) is applied after the convolution layers to introduce non-linearity, enabling the model to learn complex audio patterns. It transfers negative values to zero while preserving positive values, enhancing computational efficiency and speeding up training. It is mathematically defined as:

$$ReLU(x) = \max(0, x) \quad \dots (4)$$

This activation function helps WaveUNet adapt to the dynamic range of audio signals, improving its denoising performance in DeNoisify. [8]

4.2.2 Minimum Mean Square Error Short-Time Spectral Amplitude

The Minimum Mean Square Error Short-Time Spectral Amplitude (MMSE-STSA) algorithm is a statistical method used for speech enhancement and audio denoising, particularly as a post-processing step in systems like DeNoisify. This algorithm estimates the clean speech spectral amplitude by minimizing the mean square error, effectively suppressing noise while preserving speech characteristics. The trained model is capable of enhancing unlabeled noisy audio signals.

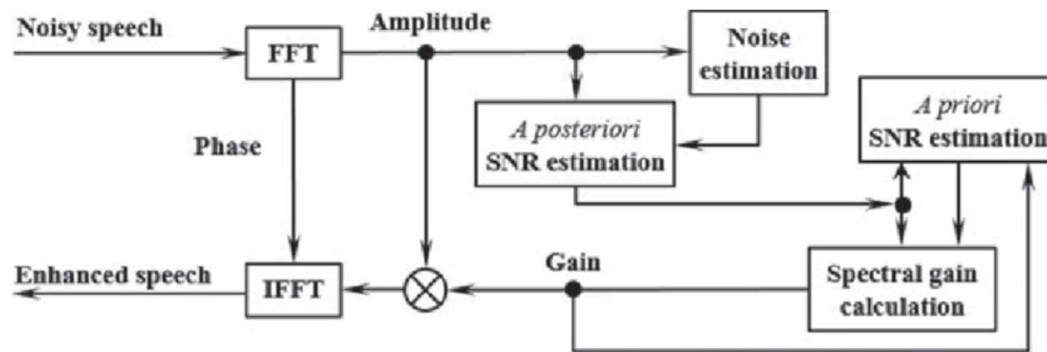


Figure 4.8: Block Diagram of MMSE-STSA

MMSE-STSA primarily involves three key components: short-time Fourier transform (STFT), noise estimation, and spectral amplitude estimation, with a focus on statistical modeling and decision-directed estimation.

- **Short-Time Fourier Transform (STFT) Layer**

This layer is the initial step in MMSE-STSA, transforming the time-domain audio signal into the frequency domain to analyze its spectral content. It applies a sliding window to the input audio signal $x(n)$ to compute the STFT $X(k, l)$, where k is the frequency bin and l is the time frame. The STFT is calculated as:

$$X(k, l) = \sum_{n=0}^{N-1} x(n) w(n - lR) e^{-j\frac{2\pi kn}{N}} \quad \dots (5)$$

Where $w(n)$ is the analysis window (e.g., Hamming window), N is the window length, and R is the hop size. This results in a time-frequency representation, and the magnitude $|X(k, l)|$ is used for further processing. The output size depends on the number of frequency bins K and time frames L , computed as $L = \lfloor (N_x - N)/R \rfloor + 1$, where N_x is the input signal length.

- **Noise Estimation Layer**

After obtaining the STFT, this layer estimates the noise spectral variance $\lambda_d(k, l)$ to distinguish it from the speech signal. It uses a noise power spectral density (PSD) estimation technique, often based on statistical averaging during speech-absent periods. The noise variance is updated recursively as:

$$\lambda_d(k, l) = \alpha \lambda_d(k, l - 1) + (1 - \alpha) |Y(k, l)|^2 \quad \dots (6)$$

Where α ($0 \leq \alpha \leq 1$) is a smoothing factor, and $Y(k, l)$ is the noisy speech STFT. This downscales the noise contribution, reducing computational complexity and making the model robust to varying noise levels.

- **Spectral Amplitude Estimation Layer**

In this layer, every frequency bin is processed to estimate the clean speech spectral amplitude $\hat{A}(k, l)$ by minimizing the mean square error. MMSE-STSA assumes a Gaussian distribution for speech and noise, using the a priori SNR (signal-to-noise ratio) and a posteriori SNR. The estimated amplitude is given by:

$$G(k, l) = \sqrt{\frac{\pi}{2}} \cdot \frac{\xi(k, l)}{1 + \xi(k, l)} \exp \left(-\frac{\gamma(k, l)}{2(1 + \xi(k, l))} \right) M! \left(1; 1; \frac{\gamma(k, l)}{2(1 + \xi(k, l))} \right) |Y(k, l)| \quad \dots (7)$$

Where: - $\xi(k, l)$ is the a priori SNR, estimated using the decision-directed approach: $\xi(k, l) = \alpha \frac{\hat{A}^2(k, l-1)}{\lambda_d(k, l)} + (1 - \alpha) \max(\gamma(k, l) - 1, 0) \gamma(k, l) = \frac{|Y(k, l)|^2}{\lambda_d(k, l)}$ is a posterior SNR. - $M(a; b; z)$ is the confluent hypergeometric function, approximating the gain function.

This results in an enhanced spectral amplitude, which is combined with the phase of $Y(k, l)$ to reconstruct the denoised signal via inverse STFT.

- **Inverse Short-Time Fourier Transform(ISTFT) Layer**

This layer is the final step in the MMSE-STSA post-processing stage of the DeNoisify system, transforming the enhanced frequency-domain representation back into the time-domain denoised audio signal for user interaction. It synthesizes the modified STFT coefficients $X(k, l)$ (with enhanced amplitude $\hat{A}(k, l)$ and original phase $\phi(k, l)$) into a time-domain signal $x(n)$, where k is the frequency bin and l is the time frame. The ISTFT is calculated as:

$$x(n) = \frac{1}{N} \sum_{l=0}^{L-1} \sum_{k=0}^{N-1} X(k, l) w(n - lR) e^{j\frac{2\pi kn}{N}} \quad \dots (8)$$

Where $w(n)$ is the synthesis window (e.g., Hamming window), N is the window length, and R is the hop size. This reconstructs the time-domain signal, with the output size depending on the number of samples N_x , computed as $N_x \approx L \cdot R + N - R$, where L is the number of time frames [9]

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation

5.1.1 Tools Used

- **Software Development Tools**

Python is the main programming language used to create the system's deep learning and core functionality. PyTorch, which offers an adaptable framework for creating and training custom neural networks, is used to implement the Wave-U-Net model and MMSE-STSA algorithm. In order to adhere to academic restrictions on library use, manual implementation was given priority where necessary, particularly for core signal processing tasks.

The user interface is built using HTML, Tailwind CSS, and JavaScript, providing a clean and responsive design for uploading audio files and interacting with the system. The interface communicates directly with the backend script executions to process audio and present denoised results.

- **Hardware Tools**

A personal laptop with a dedicated GPU was used to develop and test the entire system in order to effectively handle the Wave-U-Net model's training and inference tasks. Although the system can operate on a CPU, model execution and audio processing computation times are greatly shortened by GPU acceleration. A minimum of 8 GB RAM was sufficient to manage in-memory data operations and run the audio denoising pipeline smoothly.

- **Diagram Tools**

In this project we used StarUML as the primary diagramming tool which allowed us to create diagrams to visualize the process of construction of the system. We also used draw.io as well for creating diagrams. Using StarUML we created use case diagrams which shows the interactions and use case between different actors and the system, class diagrams which help us to model the structure of the system and activity diagrams to visualize the workflow. Using draw.io, we designed sequence diagrams to visualize the order of interactions between different objects.

- **Dataset**

For this project, the two datasets LibriSpeech and UrbanSound8K are used. To train the model, a paired dataset is created by mixing the clean signals samples from the LibriSpeech with the environmental noise samples from the UrbanSound8K. The total of 10,637 noisy-clean audio pairs is created. Among them, 7,444 pairs are allocated for the training and the remainings 3,193 pairs for the testing. Each pair contains a noisy audio file as input and its corresponding clean reference. This helps the model to learn to separate speech in realistic conditions.

5.1.2 Implementation details of Modules

- Implementation of Wave-U-Net

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=15, stride=1):
        super(ConvBlock, self).__init__()
        self.block = nn.Sequential(
            nn.Conv1d(in_ch, out_ch, kernel_size, stride, padding=kernel_size//2),
            nn.ReLU(),
            nn.Conv1d(out_ch, out_ch, kernel_size, stride, padding=kernel_size//2),
            nn.ReLU()
        )

    def forward(self, x):
        return self.block(x)

class WaveUNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, depth=5, base_ch=24):
        super(WaveUNet, self).__init__()
        self.depth = depth

        self.encoders = nn.ModuleList()
        ch = in_ch
        for i in range(depth):
            self.encoders.append(ConvBlock(ch, base_ch * (2 ** i)))
            ch = base_ch * (2 ** i)

        self.bottleneck = ConvBlock(ch, ch)

        self.decoders = nn.ModuleList()
        self.upsamplers = nn.ModuleList()
        for i in reversed(range(depth)):
            self.upsamplers.append(nn.ConvTranspose1d(ch, base_ch * (2 ** i), kernel_size=2, stride=2))
            self.decoders.append(ConvBlock(base_ch * (2 ** i) * 2, base_ch * (2 ** i)))
            ch = base_ch * (2 ** i)

        self.final = nn.Conv1d(ch, out_ch, kernel_size=1)

    def forward(self, x):
        skip_connections = []
        for encoder in self.encoders:
            x = encoder(x)
            skip_connections.append(x)
            x = F.avg_pool1d(x, kernel_size=2)

        x = self.bottleneck(x)

        for up, decoder, skip in zip(self.upsamplers, self.decoders, reversed(skip_connections)):
            x = up(x)
            if x.shape[-1] != skip.shape[-1]:
                x = F.interpolate(x, size=skip.shape[-1])
            x = torch.cat((x, skip), dim=1)
            x = decoder(x)

        return self.final(x)
```

Figure: 5.1 Implementation of Wave-U-Net Module

The Wave-U-Net module is implemented using PyTorch. This Wave-U-Net is inspired by the U-Net architecture (commonly used in image processing) but adapted for the 1D audio waveforms. This module consists of encoder blocks, a bottleneck and a decoder blocks with skip connections.

The ConvBlock class is the building block of the architecture. Each block applies two one-dimensional convolution layer and ReLU activation function between them. And, the padding is done to match the output length roughly with the input. This is done to prevent the shrinking while joining encoder and decoder.

In the first encoder block, the input is single-channel audio waveform. It applies two one-dimensional convolution layer each with 24 filter of size is 15 and followed by ReLU activation. After that, average pooling with size 2 halves the length of the sequence. In the second block, all the structure is similar but the filters is increased to 48 and pooling reduces the length. Similarly, the third block has 96 filters, the forth has 192 and the fifth block has 384 filters. The encoder is responsible for the feature extraction.

In the bottleneck the network operates on the 384 channels, and this layer applies two convolution layer with the same kernel size and ReLU activation. It is responsible to capture the high level information for the reconstruction.

In the decoder block, it simply reverses the process. It applies transposed convolution with double sequence length. After upsampling, the output is joined with the feature map of the encoder block(same level), through skip connections. The ConvBlock processes the combined features and passes them to the next stage. In case of the unmatched size, interpolation is used. This process is continued with the channel size reduced in the reversed order (384-192-96-48-24-1). Finally, 1x1 convolution layer is applied to reduce the output to one number of channels. This is the denoised of the original waveform.

- **Implementation of MMSE-STSA**

```
def mmse_stsa(x, fs, frame_len=0.032, frame_shift=0.008, alpha=0.98, beta=0.98, Gmin=0.1):
    """
    MMSE-STSA speech enhancement algorithm.
    Args:
        x: 1D numpy array, input noisy speech (normalized to [-1, 1])
        fs: sample rate
        frame_len: frame length in seconds
        frame_shift: frame shift in seconds
        alpha: smoothing factor for a priori SNR
        beta: smoothing factor for noise estimation
        Gmin: minimum gain
    Returns:
        y: enhanced speech signal
    """
    # Parameters
    nFFT = int(2 ** np.ceil(np.log2(frame_len * fs)))
    nwin = int(frame_len * fs)
    nshift = int(frame_shift * fs)
    win = np.hanning(nwin)
    x = np.append(np.zeros(nwin//2), x)
    x = np.append(x, np.zeros(nwin//2))
    nframes = int((len(x) - nwin) / nshift) + 1
    y = np.zeros(len(x))
    noise_ps = None
    prev_post_snr = None
    prev_prio_snr = None
    for i in range(nframes):
        start = i * nshift
        frame = x[start:start+nwin] * win
        X = np.fft.rfft(frame, nFFT)
        X_mag = np.abs(X)
        X_phase = np.angle(X)
        # Noise estimation (first 5 frames)
        if i < 5:
            if noise_ps is None:
                noise_ps = X_mag ** 2
            else:
                noise_ps = (noise_ps * i + X_mag ** 2) / (i + 1)
            y[start:start+nwin] += frame * win
            continue
        # Posteriori SNR
        post_snr = X_mag ** 2 / (noise_ps + 1e-12)
        # A priori SNR
        if prev_prio_snr is None:
            prio_snr = np.maximum(post_snr - 1, 0)
        else:
            prio_snr = alpha * prev_prio_snr + (1 - alpha) * np.maximum(post_snr - 1, 0)
        prev_prio_snr = prio_snr
        # Gain function (Ephraim-Malah)
        v = prio_snr * post_snr / (1 + prio_snr)
        G = (prio_snr / (1 + prio_snr)) * np.exp(-0.5 * v)
        G = np.maximum(G, Gmin)
        # Apply gain
        Y = G * X_mag * np.exp(1j * X_phase)
        y_frame = np.fft.irfft(Y, nFFT)[:nwin]
        y[start:start+nwin] += y_frame * win
        # Update noise estimate
        noise_ps = beta * noise_ps + (1 - beta) * X_mag ** 2
    # Remove padding
    y = y[nwin//2:-nwin//2]
    # Normalize
    if np.max(np.abs(y)) > 0:
        y = y / np.max(np.abs(y))
    return y
```

Figure 5.2: Implementation of MMSE-STSA

The figure 5.2 depicts the implementation of the MMSE-STSA module. Here, the parameters frame length of 0.032, frame shift 0.008, smoothing factor for priori SNR (i.e. α 0.98), and for noise estimator (i.e. β 0.98) are defined at first.

The input signal is then divided overlapping frames. For each frames, it calculates FFT to obtain its magnitude and phase. For the first few frames (here 5) ,it assumes that the frames contains only noise. Then, the noise power spectrum is estimated by averaging the frames, which is used as a baseline later. For the subsequent frames, it calculates posterior SNR (ratio of signal power to noise power) from the current frame. Then, the prior SNR is estimated using smoothing factor. This is done to blend the new information with the past SNR values, which provides a more stable estimate. The gain function is applied. This determines how much of the noisy spectrum should be kept or suppressed based on the SNR values. The minimum gain limit($G_{\min}$) is applied to avoid excessive distortion. The gain is applied to the noisy spectrum to its magnitude while keeping the original phase. This results in the enhanced frame which is transformed back into the time domain using the inverse FFT. And the frames are combined smoothly using the overlap-add. Finally, it normalizes the output signal within the $[-1, 1]$ range. The output is a cleaner version of the input audio signal.

- Training Module

```

for epoch in range(start_epoch, CONFIG["epochs"]):
    start_time = time.time()
    model.train()
    total_train_loss = 0.0
    total_train_acc = 0.0

    for noisy, clean in train_loader:
        noisy = noisy.to(CONFIG["device"])
        clean = clean.to(CONFIG["device"])
        optimizer.zero_grad()
        output = model(noisy)
        loss = criterion(output, clean)
        loss.backward()
        optimizer.step()
        total_train_loss += loss.item()

        # Normalized MSE-based accuracy proxy: 1 - (MSE / Var(target))
        with torch.no_grad():
            target_var = torch.var(clean)
            norm_mse = loss.detach() / (target_var + 1e-8)
            batch_acc = torch.clamp(1.0 - norm_mse, min=0.0, max=1.0) * 100.0
            total_train_acc += float(batch_acc)

    avg_train_loss = total_train_loss / len(train_loader)
    avg_train_acc = total_train_acc / len(train_loader)

    model.eval()
    total_val_loss = 0.0
    total_val_acc = 0.0
    with torch.no_grad():
        for noisy, clean in valid_loader:
            noisy = noisy.to(CONFIG["device"])
            clean = clean.to(CONFIG["device"])
            output = model(noisy)
            loss = criterion(output, clean)
            total_val_loss += loss.item()

            target_var = torch.var(clean)
            norm_mse = loss / (target_var + 1e-8)
            batch_acc = torch.clamp(1.0 - norm_mse, min=0.0, max=1.0) * 100.0
            total_val_acc += float(batch_acc)

    avg_val_loss = total_val_loss / len(valid_loader)
    avg_val_acc = total_val_acc / len(valid_loader)

```

Figure 5.3: Training Loop of Wave-U-Net

The module shown in Figure 5.3 is responsible for training the model. The training loop is the core process where the Wave-U-Net model learns to map noisy speech to its clean pair. It runs over the dataset in multiple iterations, known as epochs, (each epoch represents a complete pass through the training data). At the beginning of each epoch, the model is set to training mode. For every batch of data, the noisy audio is fed into the model, which predicts an enhanced version of the signal. This prediction is compared against the clean reference audio using a loss function (such as Mean Squared Error), which quantifies the difference between the two. The optimizer then updates the model parameters by performing backpropagation to minimize this loss.

5.2 Testing

Testing is the process of checking the results of the system with the expected results to make sure that the system is performing as it should.

5.2.1 Test Case for Unit Testing

Unit testing is a form of software testing that focuses on evaluating individual units or components of a software application. The goal is to ensure that each unit of the software code functions as intended. The following test scenarios were employed to assess the system after the build was completed.

Table 5.1: Test Case for User Log In

S.N.	Test Case	Input	Expected Result	Actual Result	Remarks
1	TC-LOG-1	Email:try1@gmail.com Password:Xv2jgk	Login Successful and redirect to the homepage	As Expected	Pass
2	TC-LOG-2	Email:tyy1@gmail.com Password:Gfedh3	Login fail and show the error message to enter right credentials.	As Expected	Pass

Table 5.2: Test Cases for Audio Input

S.N.	Test Cases	Test Description	Input Test Data	Expected Results	Actual Result	Remarks
1	TC-AUD-1	Validate audio file upload	.wav file	File accepted and passes to the backend	As Expected	Pass
2	TC-AUD-2	Reject Unsupported file types	Pdf, png,doc, .txt	Error and throws the error message	As Expected	Pass

Table 3.3: Test Cases for Audio Processing

S.N.	Test Cases	Test Description	Input Test Data	Expected Results	Actual Result	Remarks
1	TC-WUN-1	Process audio with Wave-U-Net	Noisy waveform	Intermediate denoised waveform returned	As Expected	Pass
2	TC-MMSE-1	Process Audio with MMSE-STSA	Output from Wave-U-Net	Final denoised waveform returned	As Expected	Pass

Table 5.4: Test Cases for Download Functionality

S.N.	Test Cases	Test Description	Input Test Data	Expected Results	Actual Results	Remarks
1	TC-DOWN-1	Download Denoised Audio	Denoised output and user is logged in	File download successfull	As Expected	Pass
2	TC-DOWN-2	Download Denoised Audio	Denoised output and user is not logged in	Unable to download and redirect the user to login	As Expected	Pass

5.2.2 Test Case for System Testing

System testing tests the entire system as a whole to ensure that the system performs as it should. It makes sure that the system is working properly and whether it can be delivered to the end users or not.

Table 5.5: Test Case for DeNoisify System

S.N.	Test Cases	Test Description	Input Test Data	Expected Results	Actual Result	Remarks
1	TC-SYS-1	End-to-end processing	Noisy .wav file	Clean audio output	As Expected	Pass
2	TC-SYS-2	Validate full pipeline sequence	Uploaded audio file	Audio processing sequentially through Wave-U-Net then MMSE-STSA	As Expected	Pass
3	TC-SYS-3	Check file upload	.wav file	File Successfully uploaded and started processing	As Expected	Pass
4	TC-SYS-4	Playback of the original audio	Uploaded audio file	Original Audio file plays successfully	As Expected	Pass
5	TC-SYS-5	Playback of the denoised audio	Output from the processing	Final Audio plays successfully	As Expected	Pass
6	TC-SYS-6	Check download functionality	Final output file	Able to download successfully	As Expected	Pass

5.3 Result Analysis

A detailed performance report was generated to evaluate the various models employed which highlights their effectiveness in accurately denoising the noisy audio waveforms. This report serves as a valuable resource for future improvements which guides further development to increase the model's efficiency and accuracy.

The dataset was tested using different hyper parameters such as different batch sizes, optimizers and learning rates.

The following table shows all the necessary information regarding the parameters of the Wave-U-Net model:

Table 5.6: Analysis Table for Wave-U-Net

Audio Length	Lr	Batch Size	No of Epochs	Network Architecture	Train Acc.	Train Loss	Val. Acc.	Val Loss
16000	0.0001	8	50	Wave-U-Net: 5 encoder-decoder blocks, Conv1D layers, skip connections	96.53%	0.00059	97.99%	0.00019
16000	0.0001	16	50	Wave-U-Net: 5 encoder-decoder blocks, Conv1D layers, skip connections	95.94%	0.00068	98.37%	0.00031

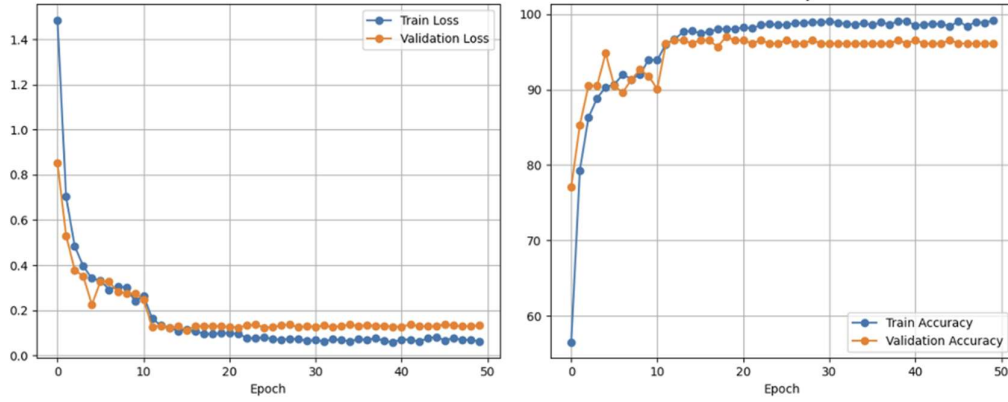


Figure 5.4: Accuracy and Loss Curve for Wave-U-Net

5.3.1 Performance Metrics

Several performance metrics are used to evaluate effectiveness of the DeNoisify system. These metrics helps to find how well the model removes the noise while preserving the original speech quality. Here, the used metrics are SNR, PESQ and STOI.

- **Signal-to-Noise Ratio (SNR)**

The SNR calculates the clarity of a speech signal by comparing the level of the output signal to the level of background noise. In the speech enhancement, higher SNR indicates greater noise reduced. It is calculated as : $SNR = 10 \log_{10}(P_{signal}/P_{noise})$ where, P_{signal} is the power of the signal, and P_{noise} is the power of the noise. A positive SNR indicates that the generated signal is greater than noise signal, otherwise the noise is present greatly on the signal.

- **Perceptual Evaluation of Speech Quality (PESQ)**

The PESQ is the standard metric for the speech quality standardized by the International Telecommunication Union (ITU-T P.862). This evaluates the perceived quality of speech signal by comparing clean reference audio with processed audio. It ranges from -0.5 to 4.5, and higher value indicated the better perceived quality.

- **Short-Time Objective Intelligibility (STOI)**

The STOI metric measures the intelligibility of a processed speech signal. It compares the processed speech time-frequency representation to a clean reference. Its value ranges from 0 to 1. The higher value represents the better quality.

Following table shows the different performance metrics of the system after processing with Wave-U-Net and postprocessing with MMSE-STSA.

Table 5.7: Summary of Performance Metrics

Metric	After Wave-U-Net	After MMSE-STSA
SNR(dB)	14.5	16.2
PESQ	2.83	2.96
STOI	0.92	0.94

The above table 5.7, shows the performance metrics of the system. After applying the Wave-U-Net, the system achieves an SNR of 14.5 dB, a PESQ of 2.83 indicating moderate perceptual quality and a STOI of 0.92 showing high speech intelligibility. After applying the MMSE-STSA as postprocessing, the system is able to achieve SNR of 16.2 dB, PESQ of 2.96 and STOI of 0.94. This shows that the performance is a bit improved after postprocessing. In overall, the system is able to improve the audio quality and provide the denoised output audio.

CHAPTER 6

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

This project successfully combined the traditional MMSE-STSA algorithm with a deep learning-based Wave-U-Net model to create a hybrid audio denoising system. The system was created to reduce undesired artifacts and improve the clarity and intelligibility of noisy speech signals.

The Wave-U-Net model proved to be highly effective in reducing non-stationary noise and learning intricate temporal features of speech. It did, however, occasionally leave behind faint background noise. By reducing any remaining noise in the spectral domain, MMSE-STSA was applied as a post-processing step to improve the denoised output from the Wave-U-Net and obtain the final denoised audio.

Numerous tests revealed that the combined strategy performed noticeably better than the individual methods. This project effectively denoises the audio signals, achieving the average SNR of 16.2 dB, PESQ of 2.96 and STOA of 0.94 across the system (i.e. after applying both Wave-U-Net and MMSE-STSA). The finished system reduced a wide variety of noise types while maintaining speech's natural quality. All things considered, this project offers a strong basis for speech enhancement applications, particularly in settings like teleconferencing, assistive technologies, voice-controlled systems, and hearing aids where clear communication is crucial.

6.2 Future Recommendation

There are tons of future recommendations that could be added to this project. They are:

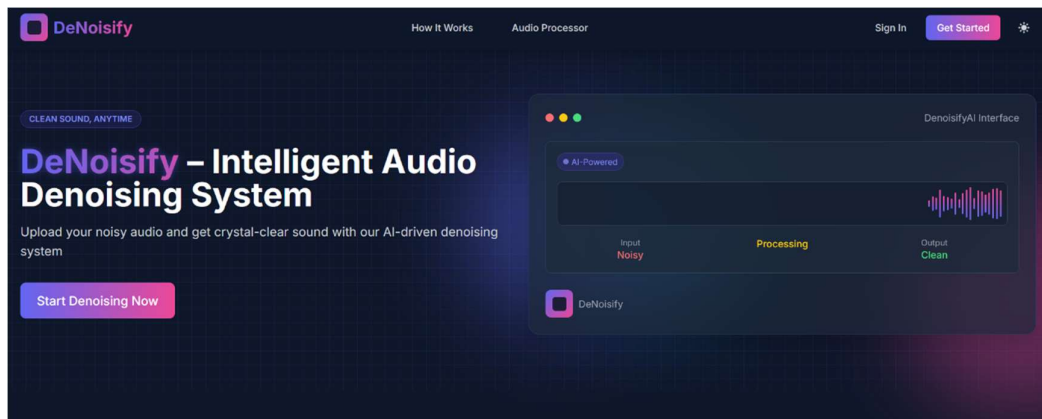
- **Real-Time Processing:** In order to enable live noise suppression in streaming or communication applications, upgrade the pipeline for real-time or near-real-time denoising.
- **Support for Multiple Formats and Sample Rates:** To increase the system's adaptability and usability, add support for more audio formats (such as MP3, FLAC), as well as different sample rates.

- **Mobile and Embedded Deployment:** Utilize quantization or pruning to maximize the model size and inference time, enabling the system to function effectively on low-power embedded systems or mobile devices.
- **User Feedback Integration:** Provide a feedback system that allows users to assign a quality rating to the output. Future iterations of the system can be improved with the help of this data.
- **Use More Diverse Audio Samples:** To increase the model's resilience and generalizability, include a more varied dataset with a range of languages, accents, and acoustic conditions.

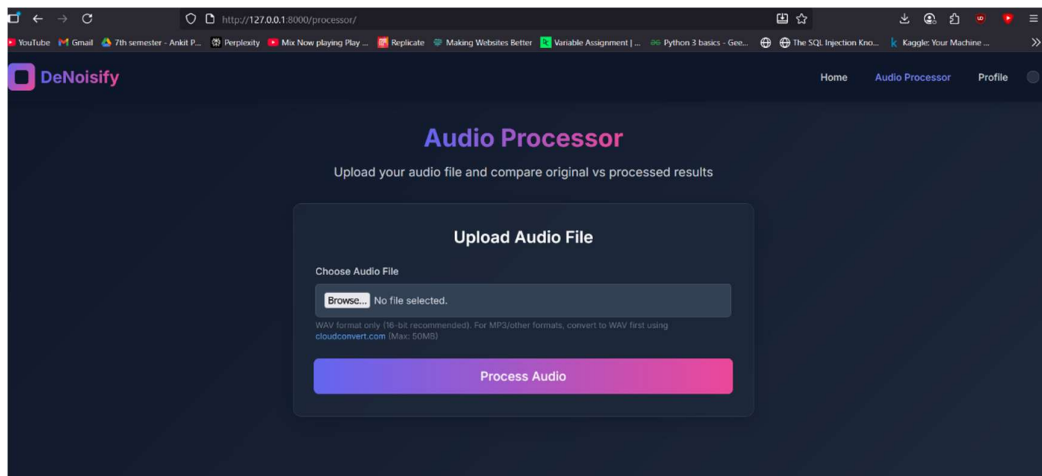
REFERENCES

- [1] Z. Kang, Z. Huang and C. Lu, "Speech Enhancement Using U-Net with Compressed Sensing," *Applied Sciences*, vol. 12, p. 4161, 2022.
- [2] Brox, Olaf Ronneberger and Philipp Fischer and Thomas, "U-Net: Convolutional Networks for Biomedical Image Segmentation," 2015.
- [3] T. Walczyna and Z. Piotrowski, "Wave-U-Net Speech Denoising," *International Business Information Management Association Computer Science Conference IBIMA*, pp. 26-27, June 2023.
- [4] C. Macartney and T. Weyde, "Improved Speech Enhancement with the Wave-U-Net," 2018.
- [5] Y. Ephraim and D. Malah, "Speech Enhancement using a minimum-mean square error short-time spectral amplitude estimator," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, pp. 1109-1121, 1984.
- [6] S. Shah and R. P. Patel, "MMSE-STSA Based Techniques for Single Channel Speech Enhancement Application," *International Journal of Innovative Science, Engineering and Technology (IJSET)*, vol. 2, pp. 140-144, 2015.
- [7] M. J. A. Khondkar, A. Ahmed, M. H. Imtiaz and S. Schuckers, "A Comparative Evaluation of Deep Learning Models for Speech Enhancement in Real-World Noisy Environments," 2025.
- [8] D. Stoller, S. Ewert and S. Dixon, "Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation," 2018.
- [9] T. Togawa, T. Otani, K. Suzuki and T. Taniguchi, "Development of speech technologies to support hearing through mobile terminal users," *APSIPA Transactions on Signal and Information Processing*, vol. 4, 2015.

APPENDICES



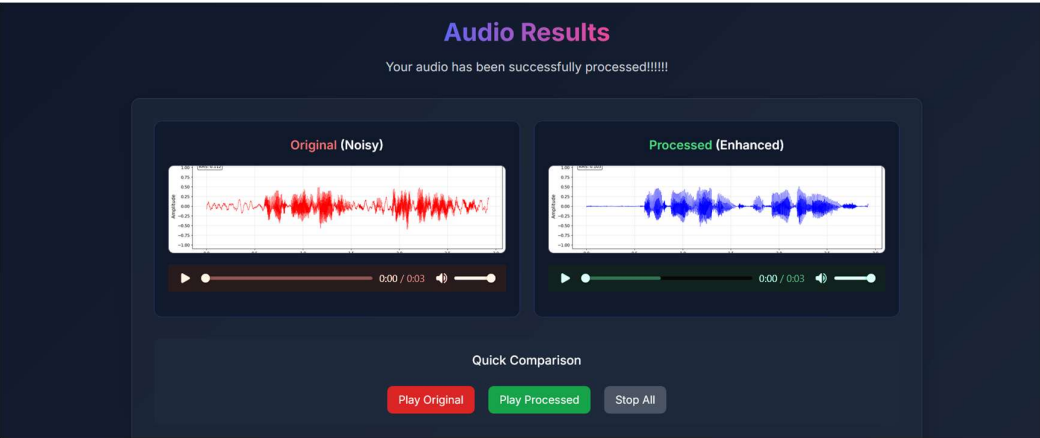
Screenshot[1]: Home Page



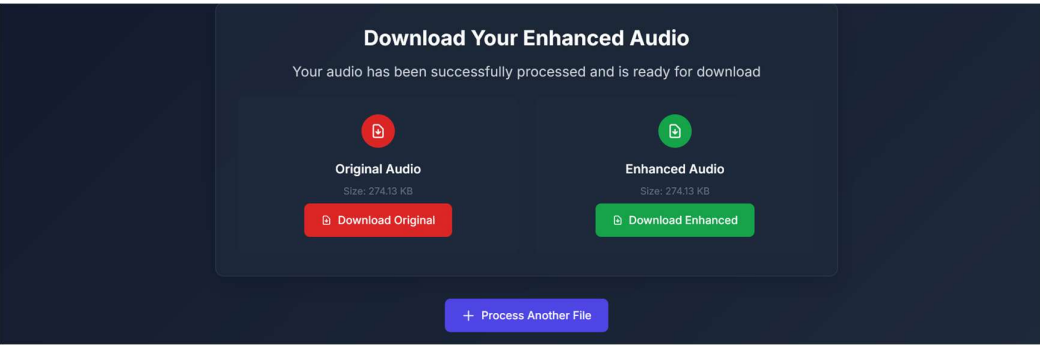
Screenshot[2]: Audio File Upload



Screenshot[3]: Audio Processing



Screenshot[4]: Audio Result Page



Screenshot[5]: Audio Download Page

Create Account

Join Denolsify to download your enhanced audio files

Username

simple

Choose a unique username for your account

Email Address

simple@gmail.com

We'll use this to send you important updates

Password

.....

Password must contain:

- At least 8 characters
- Mix of letters and numbers
- Not be too similar to your username

Confirm Password

.....

Create Account

Already have an account? [Sign in here](#)

Screenshot[6]: User Register Page

Welcome Back

Sign in to your account to download denolsed audio

Account created for simple! You can now log in.

Username or Email

simple

Password

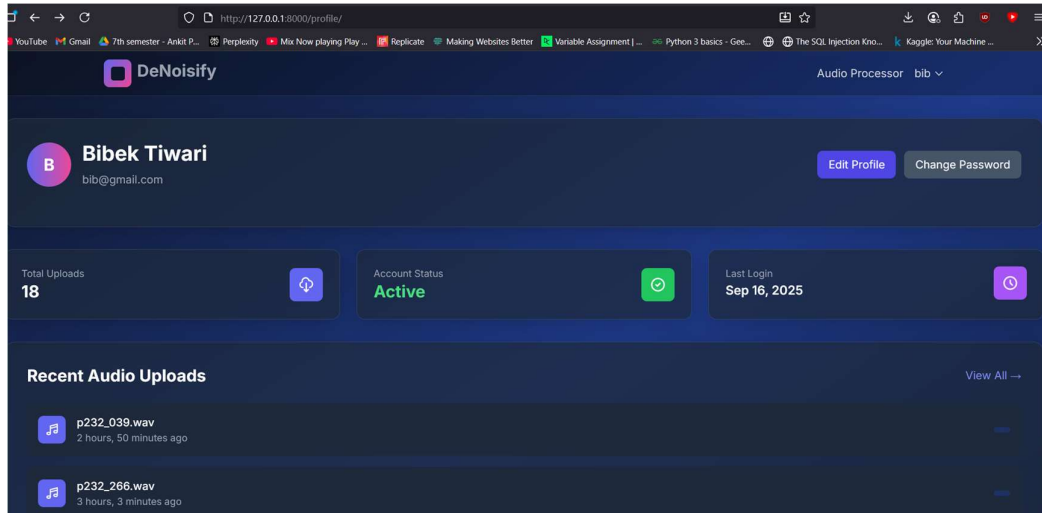
.....

Sign In

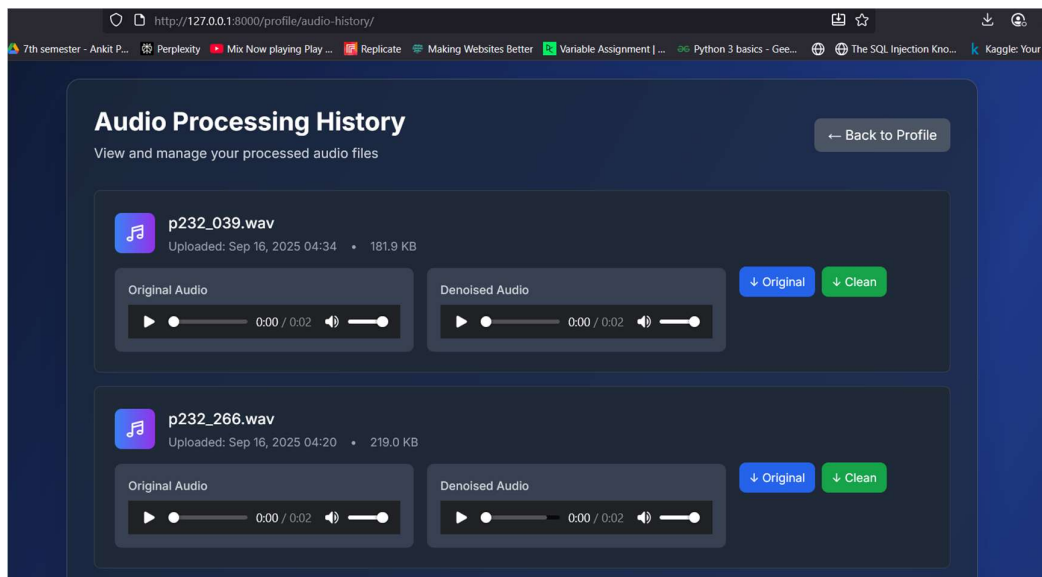
Don't have an account? [Create one here](#)

[← Back to Home](#)

Screenshot[7]: User Login Page



Screenshot[8]: User Profile Page



Screenshot[9]: Audio Processing History